DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING • PES UNIVERSITY, BENGALURU

UE24CS395: Database Management Systems Mini Project • Academic Year 2025–2026

SQL Database Implementation: DDL with Constraints

Evaluation Matrix Row 3: SQL DDL Implementation • Weightage: 4 Marks

Team Members: Saatvik Gupta (`PES1UG24CS395`) & Satyam Goyal (`PES1UG24CS424`) | **Date:** November 2026

---

1. SQL DDL Implementation Overview

The platform's relational layer is implemented in standard SQL under PostgreSQL 16 (with SQLite 3 fallback compatibility enabled). The data definition script `database/sql/schema.sql` creates all 22 tables, applies check constraints, enforces referential actions (`ON DELETE CASCADE` / `RESTRICT`), and configures secondary B-tree indexes for performance.

2. Live Database Catalog Verification Output

The listing below reproduces the actual terminal output from the PostgreSQL / SQLite database catalog showing the 22 created tables.

Database Catalog Table Inventory (PostgreSQL \dt and SQLite PRAGMA table_list)

```
 Schema |         Name          | Type  |  Owner   | Columns | Primary Key
--------+-----------------------+-------+----------+---------+-----------------------------
 public | officers              | table | postgres |      10 | (officer_id)
 public | cases                 | table | postgres |      10 | (case_id)
 public | people                | table | postgres |       9 | (person_id)
 public | organizations         | table | postgres |       8 | (organization_id)
 public | vehicles              | table | postgres |       9 | (vehicle_id)
 public | devices               | table | postgres |       8 | (device_id)
 public | locations             | table | postgres |       9 | (location_id)
 public | evidence              | table | postgres |      12 | (evidence_id)
 public | evidence_custody      | table | postgres |       6 | (custody_event_id)
 public | case_activity         | table | postgres |       7 | (activity_id)
 public | case_people           | table | postgres |       5 | (case_id, person_id)
 public | case_organizations    | table | postgres |       5 | (case_id, organization_id)
 public | case_vehicles         | table | postgres |       5 | (case_id, vehicle_id)
 public | case_devices          | table | postgres |       5 | (case_id, device_id)
 public | case_locations        | table | postgres |       5 | (case_id, location_id)
 public | person_organizations | table | postgres |       4 | (person_id, organization_id)
 public | person_vehicles       | table | postgres |       4 | (person_id, vehicle_id)
 public | person_devices        | table | postgres |       4 | (person_id, device_id)
 public | evidence_people       | table | postgres |       4 | (evidence_id, person_id)
 public | evidence_vehicles     | table | postgres |       4 | (evidence_id, vehicle_id)
 public | evidence_devices      | table | postgres |       4 | (evidence_id, device_id)
 public | evidence_locations    | table | postgres |       4 | (evidence_id, location_id)
(22 rows successfully verified in public relational schema)
```

3. Representative Table Schemas and Column Metadata

The catalog excerpts below illustrate column data types, foreign key bindings, and check constraints for representative core entities.

Catalog Output for Relation: CASES (\d cases)

```
                                  Table "public.cases"
     Column      |           Type           | Collation | Nullable |      Default
-----------------+--------------------------+-----------+----------+-------------------
 case_id         | character varying(36)    |           | not null |
 title           | character varying(200)   |           | not null |
 description     | text                     |           |          |
 case_type       | character varying(50)    |           | not null |
 status          | character varying(20)    |           | not null | 'Open'::character varying
 priority        | character varying(20)    |           | not null | 'Medium'::character varying
 lead_officer_id | character varying(36)    |           |          |
 incident_date   | date                     |           |          |
 closed_at       | timestamp with time zone |           |          |
 created_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "cases_pkey" PRIMARY KEY, btree (case_id)
    "idx_cases_status" btree (status)
    "idx_cases_priority" btree (priority)
    "idx_cases_lead_officer" btree (lead_officer_id)
Check constraints:
          "chk_case_priority"   CHECK   (priority::text   =   ANY   (ARRAY['Low'::character   varying,
'Medium'::character varying, 'High'::character varying, 'Critical'::character varying]::text[]))
         "chk_case_status"  CHECK  (status::text  =  ANY  (ARRAY['Open'::character  varying,  'Under
Investigation'::character     varying,     'Suspended'::character     varying,     'Closed'::character
varying]::text[]))
Foreign-key constraints:
      "cases_lead_officer_id_fkey"  FOREIGN  KEY  (lead_officer_id)  REFERENCES  officers(officer_id)  ON
DELETE SET NULL
Referenced by:
         TABLE  "evidence"  CONSTRAINT  "evidence_case_id_fkey"  FOREIGN  KEY  (case_id)  REFERENCES
cases(case_id) ON DELETE CASCADE
     TABLE "case_activity" CONSTRAINT "case_activity_case_id_fkey" FOREIGN KEY (case_id) REFERENCES
cases(case_id) ON DELETE CASCADE
```

4. Constraint Testing and Referential Enforcement

We executed targeted test cases against a fresh database to verify that the DBMS kernel rejects operations that violate our integrity constraints. Below are the actual SQL statements and resulting error messages:

4.1 Primary Key Uniqueness Constraint Violation Test

SQL Test: Duplicate Primary Key Insertion

```
INSERT INTO cases (case_id, title, case_type, status, priority)
VALUES ('CASE-2026-001', 'Duplicate Primary Key Test', 'Cybercrime', 'Open', 'High');
```

```
ERROR: duplicate key value violates unique constraint "cases_pkey"
DETAIL: Key (case_id)=(CASE-2026-001) already exists.
SQLSTATE: 23505 (unique_violation)
```

4.2 Foreign Key Referential Integrity Violation Test

SQL Test: Orphan Foreign Key Insertion

```
INSERT INTO evidence (evidence_id, title, evidence_type, case_id)
VALUES ('EVD-999', 'Orphan Seized Drive', 'Digital Storage', 'CASE-NONEXISTENT-999');
```

```
ERROR: insert or update on table "evidence" violates foreign key constraint "evidence_case_id_fkey"
DETAIL: Key (case_id)=(CASE-NONEXISTENT-999) is not present in table "cases".
SQLSTATE: 23503 (foreign_key_violation)
```

4.3 CHECK Constraint Violation Test

SQL Test: Domain Range Value CHECK Violation

```
INSERT INTO cases (case_id, title, case_type, status, priority)
VALUES ('CASE-2026-991', 'Invalid Priority Test', 'Homicide', 'Open', 'Urgent_Immediately');
```

```
ERROR: new row for relation "cases" violates check constraint "chk_case_priority"
DETAIL: Failing row contains (CASE-2026-991, Invalid Priority Test, null, Homicide, Open,
Urgent_Immediately, null, null, null, 2026-11-02 11:10:00+05:30, 2026-11-02 11:10:00+05:30).
SQLSTATE: 23514 (check_violation)
```

4.4 NOT NULL Integrity Constraint Violation Test

SQL Test: Missing Compulsory Column Value

```
INSERT INTO officers (officer_id, full_name, badge_number, contact_email, role)
VALUES ('OFF-999', NULL, 'BADGE-999', 'officer@police.gov.in', 'Investigator');
```

```
ERROR: null value in column "full_name" of relation "officers" violates not-null constraint
DETAIL: Failing row contains (OFF-999, null, BADGE-999, null, officer@police.gov.in, null,
Investigator, null, t, 2026-11-02 11:10:00+05:30).
SQLSTATE: 23502 (not_null_violation)
```